

Laboratory Work 2

Normalization

Task 1. Identify Normal Form Violations

1NF (First Normal Form):

The table violates 1NF because the `seat_numbers` column contains multiple values in one cell (e.g., "12A, 12B"). Each column should contain only atomic values.

2NF (Second Normal Form):

The table does not satisfy 2NF because it is not in 1NF. After converting it to 1NF, partial dependencies may occur if a composite primary key is used. For example, flight information depends on `flight_number`, not on the entire composite key.

3NF (Third Normal Form):

The table does not satisfy 3NF because it contains transitive dependencies. For example:

`booking_id` → `flight_number` → `airline_name`

Flight and airline information should be stored in separate tables to avoid redundancy.

Task 2. Step-by-step Normalization

Step 1. First Normal Form (1NF)

The `seat_numbers` column contains multiple values in one cell. To satisfy 1NF, we separate the seat numbers into individual rows.

Booking_receipt_1NF

<code>booking_id</code>	<code>passenger_full_name</code>	<code>flight_number</code>	<code>seat_number</code>
101	Anna Smith	KC101	12A
101	Anna Smith	KC101	12B
102	John Brown	KC202	15C

The remaining attributes from the original `Booking_receipt` table are retained in each corresponding row.

Result: Each cell contains only one atomic value, so the table satisfies 1NF.

Step 2. Second Normal Form (2NF)

The table must first satisfy 1NF. To achieve 2NF, we remove partial dependencies by separating booking, passenger, and flight information into different tables.

Resulting tables:

Booking

booking_id (PK)	passenger_passport_number (FK)	flight_number(FK)	ticket_price
101	P12345	KC101	50000
102	P67890	KC202	65000

Passenger

passenger_passport_number (PK)	passenger_full_name
P12345	Anna Smith
P67890	John Brown

Flight

flight_number (PK)	departure_airport_name	departure_city	arrival_airport_name	arrival_city	airline_name
KC101	Almaty Airport	Almaty	Astana Airport	Astana	Air Astana
KC202	Astana Airport	Astana	Almaty Airport	Almaty	Air Astana

Booking_seat

booking_id (PK, FK)	seat_number (PK)
101	12A
101	12B
102	15C

Result: Partial dependencies are removed by separating attributes into tables according to their functional dependencies. The tables satisfy 2NF.

Step 3. Third Normal Form (3NF)

To achieve 3NF, we remove transitive dependencies by separating airport and airline information from the Flight table.

Resulting tables:

Passenger

passenger_passport_number(PK)	passenger_full_name
P12345	Anna Smith
P67890	John Brown

Airline

airline_name (PK)
Air Astana

Airport

airport_name (PK)	city
Almaty Airport	Almaty
Astana Airport	Astana

Flight

flight_number (PK)	departure_airport_name (FK)	arrival_airport_name (FK)	airline_name (FK)
KC101	Almaty Airport	Astana Airport	Air Astana
KC202	Astana Airport	Almaty Airport	Air Astana

Booking

booking_id (PK)	flight_number (FK)	ticket_price
101	KC101	50000
102	KC202	65000

Booking_seat

booking_id (PK, FK)	seat_number (PK)	passenger_passport_number (FK)
101	12A	P12345
101	12B	P67890
102	15C	P67890

Result: Transitive dependencies are removed. Airport information is stored in the Airport table, airline information is stored in the Airline table, and passenger information is stored in the Passenger table. The resulting tables satisfy 3NF under the assumed functional dependencies.

Task 3. Functional Dependencies

The following functional dependencies justify the normalization of the original `Booking_receipt` table:

1. `booking_id` $\rightarrow$ `flight_number`, `ticket_price`

Each booking has a flight number and a ticket price.

2. `passenger_passport_number` $\rightarrow$ `passenger_full_name`

A passenger's passport number determines their full name.

3. `flight_number` $\rightarrow$ `departure_airport_name`, `arrival_airport_name`, `airline_name`

Each flight has departure and arrival airports and an airline.

4. `departure_airport_name` $\rightarrow$ `departure_city`

The departure airport determines its city.

5. `arrival_airport_name` $\rightarrow$ `arrival_city`

The arrival airport determines its city.

6. `(booking_id, seat_number)` $\rightarrow$ `passenger_passport_number`

A seat within a booking is assigned to a passenger.

Conclusion: These dependencies explain why passenger, flight, airport, airline, booking, and seat information should be stored in separate tables. This reduces data redundancy and helps prevent update, insertion, and deletion anomalies.

Task 4. Normalization of the Flights Table

Primary Key:

`flight_id` is the primary key of the `Flights` table.

Functional Dependencies:

`flight_id` $\rightarrow$ `departing_airport_id`, `arriving_airport_id`, `airline_id`

Each flight has a departure airport, an arrival airport, and an airline.

Transitive Dependencies:

Airport names and cities are stored in the separate `Airport` table, while airline information is stored in the separate `Airline` table.

The `Flights` table contains foreign keys (`departing_airport_id`, `arriving_airport_id`, `airline_id`) instead of storing airport and airline details directly.

Conclusion:

The `Flights` table satisfies 3NF with respect to these dependencies because airport and airline details are stored in separate tables. There are no transitive dependencies involving these details within the `Flights` table.

Task 5. Implementation of 3NF Tables

The normalized tables were implemented in PostgreSQL using `CREATE TABLE` statements. Primary keys and foreign keys were defined to maintain data integrity.

Sample data was inserted using `INSERT INTO` statements.

The following tables were created: `receipt_passenger`, `receipt_airline`, `receipt_airport`, `receipt_flight`, `receipt_booking`, and `receipt_booking_seat`.

All tables were successfully created and populated with sample data.

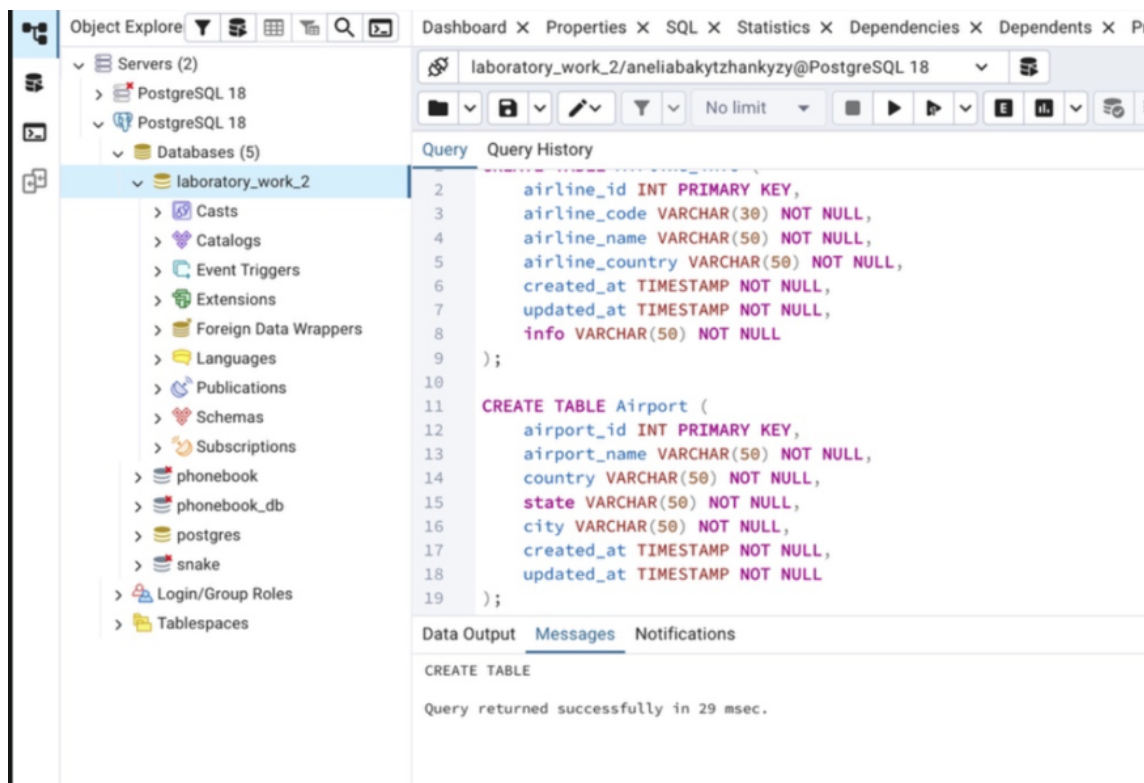
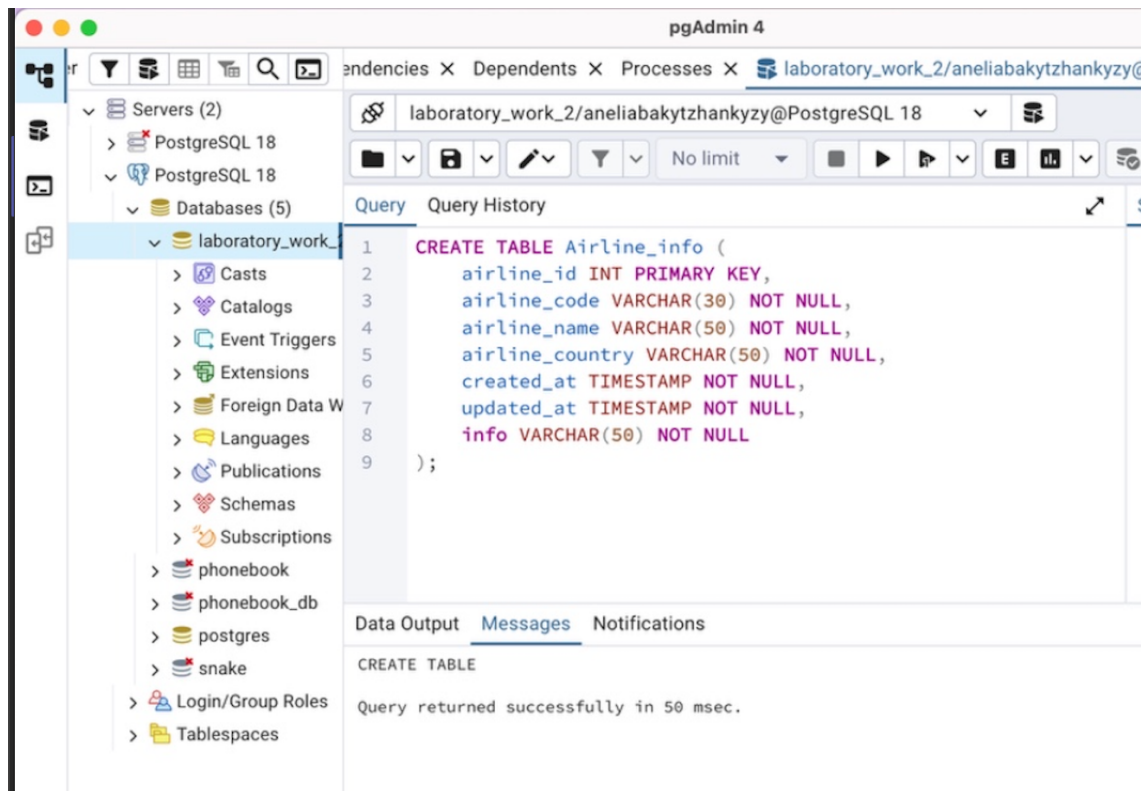
The screenshot shows the PostgreSQL IDE interface. The left pane displays the 'Object Explorer' with a tree view of the database 'laboratory_work_2'. The main pane shows a SQL query in the 'Query' editor, which is a UNION ALL query counting rows for 'Passenger', 'Airline', 'Airport', 'Flight', 'Booking', and 'Booking Seat' tables. The 'Data Output' pane at the bottom displays the results of the query as a table with two columns: 'table_name' and 'total_rows'.

table_name	total_rows
Passenger	5
Airline	2
Airport	3
Flight	3
Booking	5
Booking Seat	6

At the bottom of the IDE, a status bar indicates 'Total rows: 6' and 'Query complete 00:00:00.047'. A green message box at the bottom right states 'Successfully run. Total query runtime: 47 msec. 6 rows affected.'

DDL

1 - 2.



Object Explorer | Dashboard | Properties | SQL | Statistics | Dependencies | Dependents | Processes

Servers (2)
PostgreSQL 18
PostgreSQL 18
Databases (5)
laboratory_work_2

laboratory_work_2/aneliabakytzhankyzy@PostgreSQL 18

No limit

Query | Query History

```
11 CREATE TABLE Airport (  
12     airport_id INT PRIMARY KEY,  
13     airport_name VARCHAR(50) NOT NULL,  
14     country VARCHAR(50) NOT NULL,  
15     state VARCHAR(50) NOT NULL,  
16     city VARCHAR(50) NOT NULL,  
17     created_at TIMESTAMP NOT NULL,  
18     updated_at TIMESTAMP NOT NULL  
19 );  
20  
21 CREATE TABLE Baggage_check (  
22     baggage_check_id INT PRIMARY KEY,  
23     check_result VARCHAR(50) NOT NULL,  
24     created_at TIMESTAMP NOT NULL,  
25     updated_at TIMESTAMP NOT NULL,  
26     booking_id INT NOT NULL,  
27     passenger_id INT NOT NULL  
28 );
```

Data Output | Messages | Notifications

CREATE TABLE

Query returned successfully in 31 msec.

Object Explorer | Dashboard | Properties | SQL | Statistics | Dependencies | Dependents | Processes

Servers (2)
PostgreSQL 18
PostgreSQL 18
Databases (5)
laboratory_work_2

laboratory_work_2/aneliabakytzhankyzy@PostgreSQL 18

No limit

Query | Query History

```
19 );  
20  
21 CREATE TABLE Baggage_check (  
22     baggage_check_id INT PRIMARY KEY,  
23     check_result VARCHAR(50) NOT NULL,  
24     created_at TIMESTAMP NOT NULL,  
25     updated_at TIMESTAMP NOT NULL,  
26     booking_id INT NOT NULL,  
27     passenger_id INT NOT NULL  
28 );  
29  
30 CREATE TABLE Baggage (  
31     baggage_id INT PRIMARY KEY,  
32     weight_in_kg DECIMAL(4,2) NOT NULL,  
33     created_at TIMESTAMP NOT NULL,  
34     updated_at TIMESTAMP NOT NULL,  
35     booking_id INT NOT NULL  
36 );
```

Data Output | Messages | Notifications

CREATE TABLE

Query returned successfully in 35 msec.

Object Explorer | Dashboard | Properties | SQL | Statistics | Dependencies | Dependents | Processes

Servers (2)
PostgreSQL 18
PostgreSQL 18
Databases (5)
laboratory_work_2
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas
Subscriptions
phonebook
phonebook_db
postgres
snake
Login/Group Roles
Tablespaces

laboratory_work_2/aneliabakytzhankyzy@PostgreSQL 18

Query | Query History

```
28 );  
29  
30 CREATE TABLE Baggage (  
31     baggage_id INT PRIMARY KEY,  
32     weight_in_kg DECIMAL(4,2) NOT NULL,  
33     created_at TIMESTAMP NOT NULL,  
34     updated_at TIMESTAMP NOT NULL,  
35     booking_id INT NOT NULL  
36 );  
37  
38 CREATE TABLE Boarding_pass (  
39     boarding_pass_id INT PRIMARY KEY,  
40     booking_id INT NOT NULL,  
41     seat VARCHAR(50) NOT NULL,  
42     boarding_time TIMESTAMP NOT NULL,  
43     created_at TIMESTAMP NOT NULL,  
44     updated_at TIMESTAMP NOT NULL  
45 );
```

Data Output | Messages | Notifications

CREATE TABLE

Query returned successfully in 36 msec.

Object Explorer | Dashboard | Properties | SQL | Statistics | Dependencies | Dependents | Process

Servers (2)
PostgreSQL 18
PostgreSQL 18
Databases (5)
laboratory_work_2
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas
Subscriptions
phonebook
phonebook_db
postgres
snake
Login/Group Roles
Tablespaces

laboratory_work_2/aneliabakytzhankyzy@PostgreSQL 18

Query | Query History

```
36 );  
37  
38 CREATE TABLE Boarding_pass (  
39     boarding_pass_id INT PRIMARY KEY,  
40     booking_id INT NOT NULL,  
41     seat VARCHAR(50) NOT NULL,  
42     boarding_time TIMESTAMP NOT NULL,  
43     created_at TIMESTAMP NOT NULL,  
44     updated_at TIMESTAMP NOT NULL  
45 );  
46  
47 CREATE TABLE Booking_flight (  
48     booking_flight_id INT PRIMARY KEY,  
49     booking_id INT NOT NULL,  
50     flight_id INT NOT NULL,  
51     created_at TIMESTAMP NOT NULL,  
52     updated_at TIMESTAMP NOT NULL  
53 );
```

Data Output | Messages | Notifications

CREATE TABLE

Query returned successfully in 36 msec.

Object Explorer | Dashboard | Properties | SQL | Statistics | Dependencies | Dependents

laboratory_work_2/aneliabakytzhankyzy@PostgreSQL 18

No limit

Query Query History

```
47 CREATE TABLE Booking_flight (
48     booking_flight_id INT PRIMARY KEY,
49     booking_id INT NOT NULL,
50     flight_id INT NOT NULL,
51     created_at TIMESTAMP NOT NULL,
52     updated_at TIMESTAMP NOT NULL
53 );
54
55 CREATE TABLE Booking (
56     booking_id INT PRIMARY KEY,
57     flight_id INT NOT NULL,
58     passenger_id INT NOT NULL,
59     booking_platform VARCHAR(50) NOT NULL,
60     created_at TIMESTAMP NOT NULL,
61     updated_at TIMESTAMP NOT NULL,
62     status VARCHAR(50) NOT NULL,
63     price DECIMAL(7,2) NOT NULL
64 );
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 43 msec.

Object Explorer | Dashboard | Properties | SQL | Statistics | Dependencies | Dependents | Process

laboratory_work_2/aneliabakytzhankyzy@PostgreSQL 18

No limit

Query Query History

```
62     status VARCHAR(50) NOT NULL,
63     price DECIMAL(7,2) NOT NULL
64 );
65
66 CREATE TABLE Flights (
67     flight_id INT PRIMARY KEY,
68     sch_departure_time TIMESTAMP NOT NULL,
69     sch_arrival_time TIMESTAMP NOT NULL,
70     departing_airport_id INT NOT NULL,
71     arriving_airport_id INT NOT NULL,
72     departing_gate VARCHAR(50) NOT NULL,
73     arriving_gate VARCHAR(50) NOT NULL,
74     airline_id INT NOT NULL,
75     act_departure_time TIMESTAMP NOT NULL,
76     act_arrival_time TIMESTAMP NOT NULL,
77     created_at TIMESTAMP NOT NULL,
78     updated_at TIMESTAMP NOT NULL
79 );
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 37 msec.

Object Explore | Dashboard | Properties | SQL | Statistics | Dependencies | Dependents | Proc...

Servers (2)
PostgreSQL 18
PostgreSQL 18
Databases (5)
laboratory_work_2

- Casts
- Catalogs
- Event Triggers
- Extensions
- Foreign Data Wrappers
- Languages
- Publications
- Schemas
- Subscriptions
- phonebook
- phonebook_db
- postgres
- snake
- Login/Group Roles
- Tablespaces

laboratory_work_2/aneliabakytzhankyzy@PostgreSQL 18

No limit

Query | Query History

```
75 act_departure_time TIMESTAMP NOT NULL,  
76 act_arrival_time TIMESTAMP NOT NULL,  
77 created_at TIMESTAMP NOT NULL,  
78 updated_at TIMESTAMP NOT NULL  
79 );  
80  
81 CREATE TABLE Passengers (  
82     passenger_id INT PRIMARY KEY,  
83     first_name VARCHAR(50) NOT NULL,  
84     last_name VARCHAR(50) NOT NULL,  
85     date_of_birth DATE NOT NULL,  
86     gender VARCHAR(50) NOT NULL,  
87     country_of_citizenship VARCHAR(50) NOT NULL,  
88     country_of_residence VARCHAR(50) NOT NULL,  
89     passport_number VARCHAR(20) NOT NULL,  
90     created_at TIMESTAMP NOT NULL,  
91     updated_at TIMESTAMP NOT NULL  
92 );
```

Data Output | Messages | Notifications

CREATE TABLE

Query returned successfully in 37 msec.

Object Explore | Dashboard | Properties | SQL | Statistics | Dependencies | Dependents | Proc...

Servers (2)
PostgreSQL 18
PostgreSQL 18
Databases (5)
laboratory_work_2

- Casts
- Catalogs
- Event Triggers
- Extensions
- Foreign Data Wrappers
- Languages
- Publications
- Schemas
- Subscriptions
- phonebook
- phonebook_db
- postgres
- snake
- Login/Group Roles
- Tablespaces

laboratory_work_2/aneliabakytzhankyzy@PostgreSQL 18

No limit

Query | Query History

```
83 first_name VARCHAR(50) NOT NULL,  
84 last_name VARCHAR(50) NOT NULL,  
85 date_of_birth DATE NOT NULL,  
86 gender VARCHAR(50) NOT NULL,  
87 country_of_citizenship VARCHAR(50) NOT NULL,  
88 country_of_residence VARCHAR(50) NOT NULL,  
89 passport_number VARCHAR(20) NOT NULL,  
90 created_at TIMESTAMP NOT NULL,  
91 updated_at TIMESTAMP NOT NULL  
92 );  
93  
94 CREATE TABLE Security_check (  
95     security_check_id INT PRIMARY KEY,  
96     check_result VARCHAR(50) NOT NULL,  
97     created_at TIMESTAMP NOT NULL,  
98     updated_at TIMESTAMP NOT NULL,  
99     passenger_id INT NOT NULL  
100 );
```

Data Output | Messages | Notifications

CREATE TABLE

Query returned successfully in 40 msec.

4.

The screenshot shows a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays a tree structure of databases. The 'laboratory_work_2' database is selected, showing its contents: Casts, Catalogs, Event Triggers, Extensions, Foreign Data Wrappers, Languages, Publications, Schemas, Subscriptions, phonebook, phonebook_db, postgres, snake, Login/Group Roles, and Tablespace.

The main query editor displays the following SQL code:

```
88 country_of_residence VARCHAR(50) NOT NULL,  
89 passport_number VARCHAR(20) NOT NULL,  
90 created_at TIMESTAMP NOT NULL,  
91 updated_at TIMESTAMP NOT NULL  
92 );  
93  
94 CREATE TABLE Security_check (  
95     security_check_id INT PRIMARY KEY,  
96     check_result VARCHAR(50) NOT NULL,  
97     created_at TIMESTAMP NOT NULL,  
98     updated_at TIMESTAMP NOT NULL,  
99     passenger_id INT NOT NULL  
100 );  
101  
102 SELECT table_name  
103 FROM information_schema.tables  
104 WHERE table_schema = 'public'  
105 ORDER BY table_name;
```

Below the query editor, the 'Data Output' pane shows the results of the query, listing the table names in the 'public' schema:

table_name
airline_info
airport
baggage
baggage_ch...
boarding_pa...
booking
booking_flight
flights
security_che...

5.

The screenshot shows the DBeaver SQL editor interface. On the left, the 'Object Explorer' pane displays a tree structure of databases, with 'laboratory_work_2' selected. The main editor area shows a SQL query with line numbers 109 to 126. The query defines a table structure and then renames a table. The 'Data Output' pane at the bottom shows the execution result.

```

109     first_name VARCHAR(50) NOT NULL,
110     last_name  VARCHAR(50) NOT NULL,
111     date_of_birth DATE NOT NULL,
112     gender VARCHAR(50) NOT NULL,
113     country_of_citizenship VARCHAR(50) NOT NULL,
114     country_of_residence VARCHAR(50) NOT NULL,
115     passport_number VARCHAR(20) NOT NULL,
116     created_at TIMESTAMP NOT NULL,
117     updated_at TIMESTAMP NOT NULL
118 );
119
120 SELECT table_name
121 FROM information_schema.tables
122 WHERE table_schema = 'public'
123 ORDER BY table_name;
124
125 ALTER TABLE Airline_info
126 RENAME TO Airline;
  
```

Data Output

```

ALTER TABLE
Query returned successfully in 67 msec.
  
```

6.

The screenshot shows the DBeaver SQL editor interface. On the left, the 'Object Explorer' pane displays a tree structure of databases, with 'laboratory_work_2' selected. The main editor area shows a SQL query with line numbers 112 to 129. The query defines a table structure and then renames a table. The 'Data Output' pane at the bottom shows the execution result.

```

112     gender VARCHAR(50) NOT NULL,
113     country_of_citizenship VARCHAR(50) NOT NULL,
114     country_of_residence VARCHAR(50) NOT NULL,
115     passport_number VARCHAR(20) NOT NULL,
116     created_at TIMESTAMP NOT NULL,
117     updated_at TIMESTAMP NOT NULL
118 );
119
120 SELECT table_name
121 FROM information_schema.tables
122 WHERE table_schema = 'public'
123 ORDER BY table_name;
124
125 ALTER TABLE Airline_info
126 RENAME TO Airline;
127
128 ALTER TABLE Booking
129 RENAME COLUMN price TO ticket_price;
  
```

Data Output

```

ALTER TABLE
Query returned successfully in 27 msec.
  
```

7.

The screenshot shows the DBeaver SQL editor interface. On the left, the 'Object Explore' pane displays a tree structure of databases, with 'laboratory_work_2' selected. The main editor area shows a SQL query with line numbers 115 to 132. The query includes a table definition, a SELECT statement, and several ALTER TABLE statements. The 'Data Output' tab at the bottom shows the result of the query: 'ALTER TABLE' and 'Query returned successfully in 44 msec.'

```
115 passport_number VARCHAR(20) NOT NULL,  
116 created_at TIMESTAMP NOT NULL,  
117 updated_at TIMESTAMP NOT NULL  
118 );  
119  
120 SELECT table_name  
121 FROM information_schema.tables  
122 WHERE table_schema = 'public'  
123 ORDER BY table_name;  
124  
125 ALTER TABLE Airline_info  
126 RENAME TO Airline;  
127  
128 ALTER TABLE Booking  
129 RENAME COLUMN price TO ticket_price;  
130  
131 ALTER TABLE Flights  
132 ALTER COLUMN departing_gate TYPE TEXT;
```

ALTER TABLE
Query returned successfully in 44 msec.

8.

The screenshot shows the DBeaver SQL editor interface. On the left, the 'Object Explore' pane displays a tree structure of databases, with 'laboratory_work_2' selected. The main editor area shows a SQL query with line numbers 118 to 135. The query includes a table definition, a SELECT statement, and several ALTER TABLE statements. The 'Data Output' tab at the bottom shows the result of the query: 'ALTER TABLE' and 'Query returned successfully in 30 msec.'

```
118 );  
119  
120 SELECT table_name  
121 FROM information_schema.tables  
122 WHERE table_schema = 'public'  
123 ORDER BY table_name;  
124  
125 ALTER TABLE Airline_info  
126 RENAME TO Airline;  
127  
128 ALTER TABLE Booking  
129 RENAME COLUMN price TO ticket_price;  
130  
131 ALTER TABLE Flights  
132 ALTER COLUMN departing_gate TYPE TEXT;  
133  
134 ALTER TABLE Airline  
135 DROP COLUMN info;
```

ALTER TABLE
Query returned successfully in 30 msec.

9.

The screenshot shows the PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays a tree view of the database structure, including 'Servers (2)', 'PostgreSQL 18', and 'Databases (5)'. The 'laboratory_work_2' database is selected, showing its contents like 'Casts', 'Catalogs', 'Event Triggers', 'Extensions', 'Foreign Data Wrappers', 'Languages', 'Publications', 'Schemas', 'Subscriptions', 'phonebook', 'phonebook_db', 'postgres', 'snake', 'Login/Group Roles', and 'Tablespaces'. The main pane shows the 'Query' editor with the following SQL code:

```
123 ORDER BY table_name;
124
125 ALTER TABLE Airline_info
126 RENAME TO Airline;
127
128 ALTER TABLE Booking
129 RENAME COLUMN price TO ticket_price;
130
131 ALTER TABLE Flights
132 ALTER COLUMN departing_gate TYPE TEXT;
133
134 ALTER TABLE Airline
135 DROP COLUMN info;
136
137 ALTER TABLE Security_check
138 ADD CONSTRAINT fk_security_passenger
139 FOREIGN KEY (passenger_id)
140 REFERENCES Passengers(passenger_id);
```

The 'Messages' pane at the bottom displays an error message: 'ERROR: constraint "fk_security_passenger" for relation "security_check" already exists' and 'SQL state: 42710'.

The screenshot shows the PostgreSQL IDE interface. On the left, the 'Object Explorer' pane displays a tree view of the database structure, including 'Servers (2)', 'PostgreSQL 18', and 'Databases (5)'. The 'laboratory_work_2' database is selected, showing its contents like 'Casts', 'Catalogs', 'Event Triggers', 'Extensions', 'Foreign Data Wrappers', 'Languages', 'Publications', 'Schemas', 'Subscriptions', 'phonebook', 'phonebook_db', 'postgres', 'snake', 'Login/Group Roles', and 'Tablespaces'. The main pane shows the 'Query' editor with the following SQL code:

```
128 ALTER TABLE Booking
129 RENAME COLUMN price TO ticket_price;
130
131 ALTER TABLE Flights
132 ALTER COLUMN departing_gate TYPE TEXT;
133
134 ALTER TABLE Airline
135 DROP COLUMN info;
136
137 ALTER TABLE Security_check
138 ADD CONSTRAINT fk_security_passenger
139 FOREIGN KEY (passenger_id)
140 REFERENCES Passengers(passenger_id);
141
142 ALTER TABLE Booking
143 ADD CONSTRAINT fk_booking_passenger
144 FOREIGN KEY (passenger_id)
145 REFERENCES Passengers(passenger_id);
```

The 'Messages' pane at the bottom displays a success message: 'Query returned successfully in 36 msec.'

Object Explore | Servers (2) | PostgreSQL 18 | Databases (5) | laboratory_work_2

Query | Query History

```
133
134 ALTER TABLE Airline
135 DROP COLUMN info;
136
137 ALTER TABLE Security_check
138 ADD CONSTRAINT fk_security_passenger
139 FOREIGN KEY (passenger_id)
140 REFERENCES Passengers(passenger_id);
141
142 ALTER TABLE Booking
143 ADD CONSTRAINT fk_booking_passenger
144 FOREIGN KEY (passenger_id)
145 REFERENCES Passengers(passenger_id);
146
147 ALTER TABLE Baggage_check
148 ADD CONSTRAINT fk_baggage_check_passenger
149 FOREIGN KEY (passenger_id)
150 REFERENCES Passengers(passenger_id);
```

Data Output | Messages | Notifications

ALTER TABLE

Query returned successfully in 36 msec.

Object Explore | Servers (2) | PostgreSQL 18 | Databases (5) | laboratory_work_2

Query | Query History

```
138 ADD CONSTRAINT fk_security_passenger
139 FOREIGN KEY (passenger_id)
140 REFERENCES Passengers(passenger_id);
141
142 ALTER TABLE Booking
143 ADD CONSTRAINT fk_booking_passenger
144 FOREIGN KEY (passenger_id)
145 REFERENCES Passengers(passenger_id);
146
147 ALTER TABLE Baggage_check
148 ADD CONSTRAINT fk_baggage_check_passenger
149 FOREIGN KEY (passenger_id)
150 REFERENCES Passengers(passenger_id);
151
152 ALTER TABLE Baggage_check
153 ADD CONSTRAINT fk_baggage_check_booking
154 FOREIGN KEY (booking_id)
155 REFERENCES Booking(booking_id);
```

Data Output | Messages | Notifications

ALTER TABLE

Query returned successfully in 32 msec.

Object Explore

Servers (2)

- PostgreSQL 18
 - PostgreSQL 18
 - Databases (5)
 - laboratory_work_2
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - phonebook
 - phonebook_db
 - postgres
 - snake
 - Login/Group Roles
 - Tablespaces

laboratory_work_2/aneliabakytzhankyzy@PostgreSQL 18

No limit

Query Query History

```
143 ADD CONSTRAINT fk_booking_passenger
144 FOREIGN KEY (passenger_id)
145 REFERENCES Passengers(passenger_id);
146
147 ALTER TABLE Baggage_check
148 ADD CONSTRAINT fk_baggage_check_passenger
149 FOREIGN KEY (passenger_id)
150 REFERENCES Passengers(passenger_id);
151
152 ALTER TABLE Baggage_check
153 ADD CONSTRAINT fk_baggage_check_booking
154 FOREIGN KEY (booking_id)
155 REFERENCES Booking(booking_id);
156
157 ALTER TABLE Baggage
158 ADD CONSTRAINT fk_baggage_booking
159 FOREIGN KEY (booking_id)
160 REFERENCES Booking(booking_id);
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 37 msec.

Object Explore

Servers (2)

- PostgreSQL 18
 - PostgreSQL 18
 - Databases (5)
 - laboratory_work_2
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - phonebook
 - phonebook_db
 - postgres
 - snake
 - Login/Group Roles
 - Tablespaces

laboratory_work_2/aneliabakytzhankyzy@PostgreSQL 18

No limit

Query Query History

```
148 ADD CONSTRAINT fk_baggage_check_passenger
149 FOREIGN KEY (passenger_id)
150 REFERENCES Passengers(passenger_id);
151
152 ALTER TABLE Baggage_check
153 ADD CONSTRAINT fk_baggage_check_booking
154 FOREIGN KEY (booking_id)
155 REFERENCES Booking(booking_id);
156
157 ALTER TABLE Baggage
158 ADD CONSTRAINT fk_baggage_booking
159 FOREIGN KEY (booking_id)
160 REFERENCES Booking(booking_id);
161
162 ALTER TABLE Boarding_pass
163 ADD CONSTRAINT fk_boarding_pass_booking
164 FOREIGN KEY (booking_id)
165 REFERENCES Booking(booking_id);
```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 39 msec.

